Arduino UNO Q: WSL & USBIPD Upload Guide

Because the Arduino UNO Q is a dual-core system (a Linux processor and a Zephyr microcontroller), the upload process works differently than on a classic Arduino. The official upload tool (`remoteocd`) transfers two files over ADB: your compiled sketch payload and the Zephyr loader image `zephyr-arduino_uno_q_stm32u585xx.elf` (~2.2 MB, most of which is debug information). The board-side OpenOCD script then verifies both against the microcontroller's flash and rewrites each only if it differs.

Alternatively, you can push only the sketch payload and flash it directly using the board's on-board OpenOCD—giving you full, transparent control over the flashing and debugging workflow from WSL (over ADB or SSH).

Here is the complete method to compile, upload, and debug a sketch entirely within WSL, followed by wired/wireless IP networking and headless system configuration.

Board Architecture & Pinout Mapping

The Arduino UNO Q has a dual-chip architecture with a clear division of responsibilities:

- **Microcontroller (MCU — STM32U585):** Handles all real-time hardware I/O. **All standard Arduino Uno R3 header pins (D0–D13, A0–A5, SPI, I2C, Power/Reset) are wired directly to the STM32.**
- **Microprocessor (MPU — Qualcomm Dragonwing QRB2210):** Runs Debian Linux. It manages high-speed computing, networking, multimedia, system storage, status LEDs, and internal flashing/debugging lines.

1. Microcontroller (STM32U585) — Arduino Header Pins

The physical female headers on the edges of the board are routed to the STM32U585:

Arduino Pin	STM32 Pin	Primary / Alternate Functions (Zephyr DTS)
D0 (RX)	`PB7`	Digital I/O, UART RX
D1 (TX)	`PB6`	Digital I/O, UART TX
D2	`PB3`	Digital I/O, Timer 2 PWM (`TIM2_CH2`)
D3 (~PWM)	`PB0`	Digital I/O, Timer 3 PWM (`TIM3_CH3`)
D4	`PA12`	Digital I/O
D5 (~PWM)	`PA11`	Digital I/O, Timer 1 PWM (`TIM1_CH4`)
D6 (~PWM)	`PB1`	Digital I/O, Timer 3 PWM (`TIM3_CH4`)
D7	`PB2`	Digital I/O, Timer 8 PWM (`TIM8_CH4N`)
D8	`PB4`	Digital I/O, Timer 3 PWM (`TIM3_CH1`)
D9 (~PWM)	`PB8`	Digital I/O, Timer 4 PWM (`TIM4_CH3`)
D10 (~PWM)	`PB9`	Digital I/O, Timer 4 PWM (`TIM4_CH4`), SPI Chip Select (`SS`)
D11 (~PWM)	`PB15`	Digital I/O, Timer 1 PWM (`TIM1_CH3N`), SPI `MOSI`
D12	`PB14`	Digital I/O, Timer 1 PWM (`TIM1_CH2N`), SPI `MISO`
D13	`PB13`	Digital I/O, Timer 1 PWM (`TIM1_CH1N`), SPI `SCK` , Built-in LED
A0	`PA4`	Analog Input (`ADC1_IN9`)
A1	`PA5`	Analog Input (`ADC1_IN10`)
A2	`PA6`	Analog Input (`ADC1_IN11`)
A3	`PA7`	Analog Input (`ADC1_IN12`)
A4 (SDA)	`PC1`	Analog Input (`ADC1_IN2`), `I2C3_SDA`
A5 (SCL)	`PC0`	Analog Input (`ADC1_IN1`), `I2C3_SCL`
RESET	`NRST`	Microcontroller Hardware Reset

(The STM32 also controls onboard RGB LED channels `LED3` and `LED4`).

2. Microprocessor (Dragonwing QRB2210) — Interfaces & Peripherals

The Dragonwing Linux MPU does **not** expose general-purpose pins to the Arduino headers. Instead, it manages the board-level multimedia, communications, and power:

Interface / Peripheral	Dragonwing Assignment	Description
USB-C Port	`DWC3 USB Controller`	Power delivery, ADB access, Linux serial console (`ttyGS0`), and USB host/device
Wireless	`Qualcomm WCN3950/WCN3988`	Dual-band Wi-Fi (802.11ac) & Bluetooth 5.0 (`hci0`)
Camera Interface	`MIPI CSI (CamSS) + I2C-0`	FPC connector for camera modules
Display / Video	`MIPI DSI + ANX7625 Bridge`	Display / DP / HDMI output
Audio	`SoundWire + PM4125 Codec`	Analog audio output and microphone jack
Storage	`SDHCI / MMC0`	Onboard high-speed eMMC 5.1 storage
Hardware Buttons	`GPIO 96` (Vol Up), `GPIO 36` (Vol Down)	Fastboot, EDL recovery, and system keys
Status LEDs	`GPIO 39` (Panic Red), `GPIO 40` (WLAN Green), `GPIO 47` (BT Blue), `GPIO 41/42/60` (User RGB)	Linux system and network indicator LEDs

3. Internal Inter-Processor Interconnect (Dragonwing ↔ STM32)

The two chips communicate across internal PCB traces:

1. **High-Speed Serial Bridge (RPC Data):**
   - **Dragonwing Side:** `/dev/ttyHS1` (Qualcomm GENI High-Speed UART)
   - **STM32 Side:** `USART3` (`PB10` TX, `PB11` RX)
   - **Handshake Line:** Dragonwing `GPIO 70`
   - *Managed by the `arduino-router` daemon at 115200 baud for `Arduino_RouterBridge` RPC communication.*
2. **SWD Hardware Flash & Debug Bus:**
   - **Dragonwing `GPIO 25`** ➔ STM32 `SWDIO`
   - **Dragonwing `GPIO 26`** ➔ STM32 `SWCLK`
   - **Dragonwing `GPIO 38`** ➔ STM32 `NRST` (Hardware Reset)

- *Used by the Linux `openocd` daemon to flash sketches and provide the GDB debug server.*

Summary Rule of Thumb

- For reading sensors, toggling relays, PWM-controlling motors, or using standard Arduino shields: **Program the STM32 via Arduino C++.**
- For machine learning, computer vision, web networking, or running Python scripts: **Use the Dragonwing Linux processor and communicate with the STM32 via `Bridge` (RPC).**

Part 1: Environment Setup (Windows / WSL)

Since WSL does not natively support USB devices, you must bridge the Arduino connection from Windows to Linux. You can do this using the command line (`usbipd-win`) or a graphical interface (**WSL USB Manager**).

1. Enable USB Passthrough (Windows)

Option A: Command Line (`usbipd-win`)

1. Open Windows PowerShell as Administrator and install `usbipd-win` :

   ```
   powershell
   winget install --interactive --exact dorssel.usbipd-win
   ```

2. Plug in the Arduino UNO Q via USB and list connected devices:

   ```
   powershell
   usbipd list
   ```

3. Locate the board's `BUSID` and attach it to WSL. By adding the `--auto-attach` flag, Windows will automatically reconnect the board to WSL every time you plug it in or reset it:

   ```
   powershell
   usbipd bind --busid <BUSID>
   usbipd attach --wsl --busid <BUSID> --auto-attach
   ```

Option B: Graphical Interface (WSL USB Manager)

If you prefer a visual interface instead of typing commands every time you plug in the board, you can use **WSL USB Manager**. It acts as a graphical frontend for `usbipd` .

1. Open Windows PowerShell and install the GUI tool:

   ```
   powershell
   winget install --interactive --exact alelec.WSLUSBManager
   ```

   (Note: This automatically installs the core `usbipd-win` engine as a dependency if you don't already have it).
2. Open **WSL USB Manager** from your Windows Start Menu.
3. Plug in the Arduino UNO Q. It will appear in the list of USB devices.
4. Click the **Attach** toggle next to the device to bind it and instantly connect it to your active WSL instance.
5. **(Recommended)** Click the **Auto Attach** toggle for this device. This ensures that whenever the board reboots or is plugged in, it will automatically route straight to WSL without you having to open the manager again.

2. Configure the Linux Environment (WSL)

Before you can compile, you must install the Arduino CLI, the Zephyr core, and the Android Debug Bridge (ADB) inside your WSL instance.

Install System Packages (`adb`, `curl`, `gdb-multiarch`) Because the tools need to talk to the board's Linux processor over USB, you must install the system-wide ADB tool first. `curl` is required to download the Arduino CLI installer in the next step, and `gdb-multiarch` is needed for real-time hardware debugging in VSCode (Part 5).

```
bash
sudo apt update && sudo apt -y install adb curl gdb-multiarch
```

Install & Configure `arduino-cli` In your VSCode WSL terminal, install the CLI to a dedicated directory (e.g., `/opt/arduino-uno-q-tools/`) and install the Zephyr core:

```
bash
# Create the directory (requires sudo to write to /opt/)
sudo mkdir -p /opt/arduino-uno-q-tools

# Install the CLI binary globally
curl -fsSL https://raw.githubusercontent.com/arduino/arduino-cli/master/install.sh | sudo
BINDIR=/opt/arduino-uno-q-tools sh

# Add the custom directory to your system PATH and set the default config file location
echo 'export PATH="$PATH:/opt/arduino-uno-q-tools"' >> ~/.bashrc
echo 'export ARDUINO_CONFIG_FILE="/opt/arduino-uno-q-tools/arduino-cli.yaml"' >> ~/.bashrc
source ~/.bashrc

# Create ecosystem directories and grant write permissions across the folder
sudo mkdir -p /opt/arduino-uno-q-tools/{data,downloads,user}
sudo chmod -R a+w /opt/arduino-uno-q-tools

# Initialize config directly into /opt/arduino-uno-q-tools/ and configure directory paths
arduino-cli config init --dest-dir /opt/arduino-uno-q-tools --overwrite
arduino-cli config set directories.data /opt/arduino-uno-q-tools/data
arduino-cli config set directories.downloads /opt/arduino-uno-q-tools/downloads
arduino-cli config set directories.user /opt/arduino-uno-q-tools/user

# Optional fallback: link to default location for tools that do not source ~/.bashrc
mkdir -p ~/.arduino15 && ln -sf /opt/arduino-uno-q-tools/arduino-cli.yaml
~/.arduino15/arduino-cli.yaml

# Install the Zephyr core (files will be written to /opt/ by the normal user)
arduino-cli core update-index
arduino-cli core install arduino:zephyr
```

Verify ADB Connection: Verify your board is successfully bridged by running `adb devices` in WSL. You should see an alphanumeric serial number (e.g., `1621644798 device`).

> **TIP**
>
> **No Permissions / Missing Udev Rules Error?** If `adb devices` throws a `no permissions` error, it is because WSL doesn't always run the background `udev` permissions daemon properly. The easiest fix is to kill the ADB server and restart it as `root` so it can access the USB ports directly:
>
> ```
> bash
> sudo adb kill-server && sudo adb start-server
> adb devices
> ```

Part 2: Compilation

1. Create the Sketch File

Arduino sketches require the main `.ino` file to be inside a folder of the exact same name. Let's create a `led` folder and place your `led.ino` code inside it using the terminal:

```
bash
mkdir -p led
cat << '__EOF__' > led/led.ino
#include <Arduino.h>

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(50); // Simple software debounce
  digitalWrite(LED_BUILTIN, LOW);
  delay(50); // Simple software debounce
}
__EOF__
```

2. Understanding Link Modes

When you compile a sketch for the UNO Q, `arduino-cli` does not produce a standalone firmware. The microcontroller always runs a pre-built **Zephyr loader** (the "bootloader" firmware shipped with the core). Your sketch is built as an add-on that this loader starts. *How* the sketch is joined to the loader is controlled by the board's **Link mode** option.

Both modes start from the same compiled object files. They differ in the final link step, in the file you upload, and in what the loader has to do at boot.

Dynamic (default)

The sketch is built as a **Zephyr LLEXT (Linkable Loadable Extension)** — conceptually similar to a shared library (`.so`) on Linux.

1. **Compile-time check:** The build first does a trial link against `syms-dynamic.ld` , a list of the symbols the loader exports (Arduino API, libc, kernel calls, devices). This catches undefined symbols and size overflows during compilation instead of at boot.

2. **Partial link:** The real link uses `ld -r`, producing a *relocatable* ELF object. Function calls into the loader are left unresolved, and no final addresses are assigned (the `.text` section starts at `0x00000000`).
3. **Packaging:** `zephyr-sketch-tool` stamps a small header into the ELF's spare identification bytes (the file length plus Arduino's `0x2341` marker) and saves it as `<sketch>.ino.elf-zsk.bin`.
4. **At boot:** The loader reads that ELF directly from flash at `0x08100000` using Zephyr's LLEXT subsystem. It allocates memory from a dedicated heap (`CONFIG_LLEXT_HEAP_SIZE=256` KB), copies the code into **RAM**, resolves each symbol **by name** against its export table, applies relocations, and calls `setup()` / `loop()`. Read-only data is kept in flash (`CONFIG_LLEXT_RODATA_NO_RELOC`).

Trade-offs:

- ✅ **Tolerates loader rebuilds:** Symbols are matched by name, so the sketch keeps working if the loader is rebuilt and functions move to different addresses (as long as the exported API is unchanged).
- ✅ **Default and best supported:** This is what Arduino ships and tests.
- ❌ **Larger payload:** The file carries relocation and symbol tables (the LED sketch is ~74 KB).
- ❌ **Limited API:** Only symbols the loader explicitly exports can be called.
- ❌ **Code runs at boot-assigned RAM addresses:** The code's real addresses aren't known until the loader places it. This is why source-level breakpoints failed during earlier debugging attempts: the program was running at RAM addresses like `0x20015f58`, while the `.elf` listed its functions near `0x00000000`.

Static

The sketch is **fully linked at build time** against fixed addresses inside the loader firmware.

1. **Full link:** The linker uses `syms-static.ld` (the absolute address of *every* public symbol in the loader, roughly twice as many as the dynamic list, including STM32 HAL and compiler-runtime functions), plus `memory-static.ld` and `build-static.ld`. There is no `-r`, so every call is resolved to a hard-coded address.
2. **Packaging:** The linked image is converted to a raw `.bin`, then `zephyr-sketch-tool -prelinked` ("Provided file has already been linked to Zephyr") adds its header, producing `<sketch>.ino.bin-zsk.bin`.
3. **At boot:** No relocation or symbol lookup is needed. The loader can start the pre-linked image directly.

Trade-offs:

- ✅ **Enables real-time hardware debugging:** Because the compiler generates an executable ELF (`EXEC`) with fixed flash addresses (`0x08100000`+) rather than a relocatable object (`ET_REL` at

`0x00000000`), GDB and OpenOCD can plant hardware breakpoints directly on `setup()` and `loop()` and step through your source code line-by-line!

- ✅ **Smaller payload and faster start:** No symbol or relocation tables, and no load-time linking (LED sketch payload is 24,788 bytes vs 75,488 bytes).
- ✅ **Wider API:** It can call symbols the dynamic export list doesn't include.
- ❌ **Tied to one exact loader build:** Both symbol files say they were generated from a specific `zephyr.elf` (identified by a SHA256 hash). If the loader on the board isn't *exactly* that build, the hard-coded addresses point at the wrong code and the sketch will crash or behave unpredictably. This matters for the manual upload method in Part 3, which **never refreshes the loader**. *(Note: In physical board verification, the pre-installed loader on the UNO Q matched the core's loader with 0 code differences).*
- ❌ **Less common path:** It's an option rather than the default, so it gets less real-world use.

Side-by-side

	Dynamic (default)	Static
FQBN	`arduino:zephyr:unoq`	`arduino:zephyr:unoq:link_mode=static`
Final link	Relocatable (`ld -r`)	Fully linked at fixed addresses
Symbol file used	`syms-dynamic.ld` (exported API only)	`syms-static.ld` (all loader symbols)
File to upload	`<sketch>.ino.elf-zsk.bin`	`<sketch>.ino.bin-zsk.bin`
`zephyr-sketch-tool` flag	*(none)*	`-prelinked`
Work done at boot	Load into RAM, resolve symbols by name, relocate	None, starts the pre-linked image
Must the loader match exactly?	No, only its exported API	**Yes**, byte-for-byte
Hardware Breakpoints	❌ Fails (dynamic RAM address)	✅ **Works!** (Fixed Flash address)

> **TIP**
>
> **Which should I use?** Stay with **Dynamic** unless you have a specific reason not to (such as needing hardware breakpoint debugging with GDB). Its main weakness (a larger payload) rarely matters, and it keeps working when the loader on the board and your installed core drift slightly apart.

3. Compile the Sketch

From the directory containing your new `led` folder, compile the sketch choosing one of the two link modes:

Option A: Dynamic Mode (Default)

Dynamic is used by default when no link mode flag is specified. It produces a relocatable extension (`.elf-zsk.bin`) that the Zephyr loader links at boot time:

```
bash
arduino-cli compile -b arduino:zephyr:unoq --output-dir ./led/build ./led
```

- **Upload Artifact:** `./led/build/led.ino.elf-zsk.bin` (~74 KB)

Option B: Static Mode (Required for Hardware Breakpoints & Debugging)

To compile in Static mode, append `:link_mode=static` to the board FQBN. This resolves all symbols at build time to fixed flash addresses:

```
bash
arduino-cli compile -b arduino:zephyr:unoq:link_mode=static --output-dir ./led/build ./led
```

- **Upload Artifact:** `./led/build/led.ino.bin-zsk.bin` (~24 KB)

> IMPORTANT
>
> **Upload the file that matches the mode you compiled with:**
>
> - **Dynamic:** upload `led.ino.elf-zsk.bin`.
> - **Static:** upload `led.ino.bin-zsk.bin`.
>
> **Both files are created in either mode**, so finding a file in `build/` doesn't make it valid. In a Dynamic build, the `.bin-zsk.bin` is just a raw dump of an unlinked object and **will not run**. In a Static build, the `.elf-zsk.bin` is not the upload artifact. If you switch between modes, clean the `build/` folder (`rm -rf ./led/build`) before recompiling to avoid flashing a stale payload.

- **Why `--output-dir`?** By default, `arduino-cli` places compiled binaries in a cache folder under your home directory (`~/.cache/arduino/sketches/...`, or `/root/.cache/...` if run as root). This flag copies the output into `./led/build/`, making it easy to grab the correct upload file.

Part 3: The Manual Upload (Direct OpenOCD Flashing)

Instead of `arduino-cli upload`, we push only the sketch payload to the board and flash it using the board's internal OpenOCD. The Zephyr loader already on the microcontroller is left untouched.

> NOTE
>
> This method assumes the Zephyr loader on the board is already present and compatible with your installed core version. The official tool re-flashes the loader when it differs; this manual method never does. If you update the `arduino:zephyr` core and sketches stop running, perform one official upload (or burn the bootloader) to refresh the loader.

Extract the Board Serial: Run this command to automatically parse your board's serial number into an environment variable so the subsequent commands can use it:

```
bash
export ADB_SERIAL=$(adb devices | grep -w device | cut -f1)
```

1. Create a Staging Folder on the Board Use ADB to create a temporary folder on the board's Linux side.

```
bash
adb -s ${ADB_SERIAL} shell mkdir -p /tmp/remoteocd
```

2. Push the Sketch Payload Push only your compiled sketch payload to the board:

```
bash
# If compiled with Dynamic mode (default):
adb -s ${ADB_SERIAL} push ./led/build/led.ino.elf-zsk.bin /tmp/remoteocd/sketch.bin

# If compiled with Static mode:
# adb -s ${ADB_SERIAL} push ./led/build/led.ino.bin-zsk.bin /tmp/remoteocd/sketch.bin
```

> NOTE
>
> **How big is the payload?** There is no 100 KB limit. For reference, the LED blink sketch above produces a **~74 KB** `.elf-zsk.bin`. Sketches that pull in libraries (e.g., `Arduino_RouterBridge`) will be larger. The real limits, defined by the core's `boards.txt`, are **768 KB (786,432 bytes) of flash** and **256 KB (262,144 bytes) of RAM** — `arduino-cli compile` prints your usage against these after every build. You can check the payload size with `ls -l ./led/build/*.elf-zsk.bin`.

3. Flash the Microcontroller Run the board's internal OpenOCD to flash the STM32 microcontroller directly.

```
bash
adb -s ${ADB_SERIAL} shell "/opt/openocd/bin/openocd -d2 -s /opt/openocd -s
/opt/openocd/share/openocd/scripts -f openocd_gpiod.cfg -c 'init' -c 'reset halt' -c 'flash
write_image erase /tmp/remoteocd/sketch.bin 0x8100000 bin' -c 'reset' -c 'sleep 100' -c
'mww 0x40036400 0xCAFFEEEE' -c 'shutdown'"
```

Understanding the Flash Command

Because the Arduino UNO Q has two processors (a Linux processor and an STM32 microcontroller), this command runs *on the Linux processor* to directly program the STM32. The sequence mirrors the sketch-flashing steps of the core's official `flash_sketch.cfg` script:

- `/opt/openocd/bin/openocd`: The OpenOCD executable residing on the board's Linux filesystem.
- `-d2`: Sets the debug output level to `2` (Info). This suppresses overly verbose debugging logs while still showing critical success/failure messages.
- `-s /opt/openocd ...`: The `-s` flags add specific directories to OpenOCD's search path so it can locate necessary scripts and configuration files.
- `-f openocd_gpiod.cfg`: Loads the board-specific hardware interface configuration. This tells OpenOCD how the Linux processor should use its GPIO pins to "bitbang" the SWD (Serial Wire Debug) protocol to communicate with the STM32 microcontroller.
- `-c 'init'`: Initializes OpenOCD, connects to the STM32 via SWD, and prepares it for commands.
- `-c 'reset halt'`: Resets the STM32 and immediately halts its CPU so the flash can be written from a known, stable state.
- `-c 'flash write_image erase ... 0x8100000 bin'`: Erases the required sectors and writes the sketch payload starting at `0x08100000` — the start of the STM32U585's second flash bank, which is the core's configured sketch address (`unoq.upload.address`). The Zephyr loader lives below this address and is not touched.
- `-c 'reset'`: Resets the STM32 so it starts from the updated flash.
- `-c 'sleep 100'`: Pauses for 100 milliseconds to let the reset cycle progress.
- `-c 'mww 0x40036400 0xCAFFEEEE'`: "Memory Write Word" — writes the value `0xCAFFEEEE` to address `0x40036400`. The official script performs this exact write after flashing a sketch.
- `-c 'shutdown'`: Closes OpenOCD and returns control to your WSL terminal.

> NOTE

> **What is confirmed vs. inferred about `0xCAFFEEEE`:** It is confirmed that the official `flash_sketch.cfg` writes this value to this address after flashing. The *explanation* that it is a "magic word" telling the Zephyr loader to start the newly flashed sketch is an inference and has not been verified against the loader source.

> NOTE
>
> The official `flash_sketch.cfg` also sets `reset_config srst_only srst_nogate srst_push_pull connect_assert_srst` before `init`. The command above does not set it explicitly. If you ever see unreliable connects or resets, adding `-c 'reset_config srst_only srst_nogate srst_push_pull connect_assert_srst'` before `-c 'init'` matches the official behaviour more closely.

Part 4: Direct Wired IP Networking (Ethernet-over-USB via CDC-NCM)

As an alternative to virtual USB bridging (`usbipd-win`), you can configure the UNO Q's USB-C port to act as an **Ethernet-over-USB gadget**.

This gives you the ultimate development experience: **a single USB-C cable supplies 5V power AND a direct high-speed TCP/IP network link.**

- Windows natively recognizes the board as a USB Network Adapter without custom drivers.
- WSL accesses the board directly over local IP without USB passthrough.
- You can compile, flash, debug, and log in via `ssh` and `scp` with zero wireless latency, even with no Wi-Fi router.

The Dragonwing Linux kernel already includes the compiled kernel modules: `usb_f_ncm.ko` (modern CDC-NCM standard), `usb_f_rndis.ko`, and `usb_f_ecm.ko`.

Step 1: Add the NCM Function to the USB Gadget Script

The USB-C port is managed by `/usr/lib/android-sdk/platform-tools/adbd-usb-gadget`.

Edit the script on the board:

```
bash
adb shell sudo nano /usr/lib/android-sdk/platform-tools/adbd-usb-gadget
```

1. Locate the `setup()` function (around line 75) where `functions/acm.GS0` is added:

```sh
    # create the acm (serial) function with unique name
    mkdir -p functions/acm.GS0
    # assign acm function to configuration
    ln -s functions/acm.GS0 configs/c.1
```

Directly below those lines, add:

```sh
    # create the USB Ethernet (NCM) function
    modprobe usb_f_ncm
    mkdir -p functions/ncm.usb0
    ln -s functions/ncm.usb0 configs/c.1
```

2. Locate the `reset()` function (around line 95) and add the cleanup line:

```sh
    rm -f ${CONFIGFS_DIR}/configs/c.1/ncm.usb0
```

Save and exit (`Ctrl + O`, `Enter`, `Ctrl + X`).

Step 2: Configure NetworkManager for `usb0` (Static IP & DHCP)

Configure NetworkManager on the board to assign the board IP `192.168.7.1` and automatically serve an IP (e.g. `192.168.7.2`) to your PC:

```bash
adb shell "sudo tee /etc/NetworkManager/system-connections/usb0.nmconnection << 'EOF'
[connection]
id=usb0
type=ethernet
interface-name=usb0

[ipv4]
address1=192.168.7.1/24
method=shared

[ipv6]
method=ignore
EOF"

adb shell sudo chmod 600 /etc/NetworkManager/system-connections/usb0.nmconnection
adb shell sudo nmcli connection reload
```

Step 3: Reboot the Board

```
bash
adb shell sudo reboot
```

Once rebooted, Windows will detect a standard USB Ethernet device. From WSL or PowerShell, you can immediately reach the board at `192.168.7.1` or `uno-q.local`.

Part 5: Wireless IP Networking (Wi-Fi & SSH without App Lab)

If you prefer to work completely wire-free, the Dragonwing Debian OS uses standard **NetworkManager** (`nmcli`) and **OpenSSH**. You do not need the proprietary Arduino App Lab software.

Step 1: Connect to Wi-Fi via `nmcli`

Run these commands from your WSL terminal (or via ADB/USB shell):

1. **Scan available Wi-Fi networks:**

   ```
   bash
   adb shell nmcli dev wifi
   ```

2. **Connect to your Wi-Fi network:**

   ```
   bash
   adb shell nmcli dev wifi connect "YOUR_WIFI_SSID" password "YOUR_WIFI_PASSWORD"
   ```

3. **Verify the connection and inspect the board's IP address:**

   ```
   bash
   adb shell ip addr show wlan0
   ```

   Look for the `inet` line (e.g., `inet 192.168.1.125/24`).

Step 2: Set a Password & Enable SSH

1. **Set a password for the `arduino` user:**

```
bash
adb shell "echo -e 'YOUR_CHOSEN_PASSWORD\nYOUR_CHOSEN_PASSWORD' | sudo
/usr/local/bin/arduino-passwd"
```

2. **Enable and start the SSH server:**

```
bash
adb shell sudo systemctl enable --now ssh
```

3. **(Recommended) Enable mDNS (Avahi) discovery:**

```
bash
adb shell sudo systemctl enable --now avahi-daemon
```

(This allows addressing the board as `uno-q.local` instead of tracking dynamic IP addresses).

Standalone Hotspot Mode (No Router)

If you are away from a router, turn the Arduino UNO Q into its own standalone Wi-Fi Access Point:

```
bash
adb shell nmcli dev wifi hotspot ifname wlan0 ssid "Arduino-Uno-Q" password "arduino123"
```

Join the `Arduino-Uno-Q` Wi-Fi network from your PC and connect via SSH to `arduino@10.42.0.1`.

Part 6: VSCode Integration via SSH (Unified Tasks & Hardware Debugging)

Now that the board is accessible over standard IP (either via **Ethernet-over-USB** at `192.168.7.1` or via **Wi-Fi** at `uno-q.local`), you can replace ADB entirely with standard `ssh` and `scp`.

1. Configure Passwordless SSH (One-Time Setup)

In your WSL terminal, copy your SSH public key to the board so tasks and debugging run without password prompts:

```
bash
# If you connect via Wi-Fi:
ssh-copy-id arduino@uno-q.local

# If you connect via Ethernet-over-USB:
ssh-copy-id arduino@192.168.7.1
```

2. Configure VSCode Tasks (`.vscode/tasks.json`)

Create or update `.vscode/tasks.json` in your workspace. This configuration compiles in **Static link mode** and uses `scp` and `ssh` to flash the microcontroller:

```
json
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "Arduino UNO Q: Compile Current Sketch",
            "type": "shell",
            "command": "/opt/arduino-uno-q-tools/arduino-cli compile -b
arduino:zephyr:unoq:link_mode=static --build-path \"${fileDirname}/build-static/tmp\" --
output-dir \"${fileDirname}/build-static\" \"${fileDirname}\"",
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "presentation": {
                "reveal": "always",
                "panel": "new"
            },
            "problemMatcher": []
        },
        {
            "label": "Arduino UNO Q: Upload via SSH",
            "type": "shell",
            "command": "/opt/arduino-uno-q-tools/arduino-cli compile -b
arduino:zephyr:unoq:link_mode=static --build-path \"${fileDirname}/build-static/tmp\" --
output-dir \"${fileDirname}/build-static\" \"${fileDirname}\" && scp
\"${fileDirname}/build-static/\"*.bin-zsk.bin arduino@uno-q.local:/tmp/remoteocd/sketch.bin
&& ssh arduino@uno-q.local \"/opt/openocd/bin/openocd -d2 -s /opt/openocd -s
/opt/openocd/share/openocd/scripts -f openocd_gpiod.cfg -c 'init' -c 'reset halt' -c 'flash
write_image erase /tmp/remoteocd/sketch.bin 0x8100000 bin' -c 'reset' -c 'sleep 100' -c
'mww 0x40036400 0xCAFFEEEE' -c 'shutdown'\"",
            "group": {
                "kind": "test",
                "isDefault": true
            },
            "presentation": {
                "reveal": "always",
                "panel": "new"
            },
            "problemMatcher": []
        }
    ]
}
```

(Note: If using Ethernet-over-USB instead of Wi-Fi, simply change `arduino@uno-q.local` to `arduino@192.168.7.1`).

3. Launch the OpenOCD Debug Server via SSH Tunnel

Before attaching the VSCode debugger, tunnel OpenOCD's GDB port (`3333`) over SSH from the board to your local machine.

In a **separate WSL terminal window**, run:

```
bash
ssh -L 3333:localhost:3333 -N arduino@uno-q.local "/opt/openocd/bin/openocd -d2 -s
/opt/openocd -s /opt/openocd/share/openocd/scripts -c 'proc gdb_attach_hook {} { halt }' -c
'proc gdb_detach_hook {} { resume }' -f openocd_gpiod.cfg"
```

Leave this terminal window running while debugging.

4. Create the VSCode Debug Configuration (`.vscode/launch.json`)

Configure `.vscode/launch.json` to attach to the SSH-tunneled GDB server. Because Static mode was used, symbols map directly to physical flash:

```
json
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Arduino UNO Q: Hardware Debug (SSH)",
            "cwd": "${workspaceFolder}/led",
            "executable": "${workspaceFolder}/led/build-static/led.ino_debug.elf",
            "request": "attach",
            "type": "cortex-debug",
            "servertype": "external",
            "gdbTarget": "localhost:3333",
            "gdbPath": "/usr/bin/gdb-multiarch",
            "showDevDebugOutput": "raw",
            "postAttachCommands": []
        }
    ]
}
```

5. Debugging Workflow

1. Run the **Arduino UNO Q: Upload via SSH** task (`Ctrl + Shift + P` -> **Tasks: Run Task**).
2. Ensure the OpenOCD SSH tunnel terminal from Step 3 is running.
3. Open `led/led.ino` and set a breakpoint by clicking in the left margin next to a line inside `loop()` or `setup()`.
4. Press `F5` (or click the green Play button in the Run & Debug view).
5. VSCode attaches to the target over the network. Execution halts immediately at your breakpoint! You can inspect local and global variables, view call stacks (`loop()` called from `main()`), and use Step Over (`F10`) or Step Into (`F11`) to walk through your code line-by-line.

Part 7: Headless Optimization (Disabling the Graphical Desktop)

By default, the Dragonwing Debian OS boots into `graphical.target` and runs **LightDM**, **Xorg**, and a **GTK Greeter** in the background, consuming **~250 MB to 300 MB of RAM** even when no physical screen is attached.

Switching to a clean, headless text-only server mode (`multi-user.target`) frees up substantial system memory and reduces heat/idle CPU load.

1. Disable the GUI and Switch to Headless Mode

Open an interactive shell via ADB:

```
bash
adb shell
```

Then run:

```
bash
# 1. Set the default boot target to headless (text-only / multi-user)
sudo systemctl set-default multi-user.target

# 2. Disable LightDM so the display manager does not start on boot
sudo systemctl disable lightdm

# 3. Stop LightDM and Xorg immediately (frees RAM without requiring a reboot)
sudo systemctl stop lightdm
```

(Note: If you have not set a password for the `arduino` user yet, run Step 2 from Part 5 first to configure your password).

2. What Changes?

- **RAM Freed:** ~250–300 MB of RAM is instantly reclaimed (no Xorg display server, GTK greeter, or desktop audio session daemons).
- **CPU & Power:** Idle CPU load drops and the board runs noticeably cooler.
- **What keeps working:** Everything essential—**ADB**, **SSH**, **Wi-Fi / NetworkManager**, **Docker**, **Python**, and the **Arduino Router** RPC bridge—runs under `multi-user.target` and remains fully functional.

3. Restoring the GUI (Rollback)

If you ever plug in a display shield or HDMI/DSI monitor and need the graphical desktop back, run:

```
bash
sudo systemctl set-default graphical.target
sudo systemctl enable --now lightdm
```

Appendix: Verified Facts

Values below were read and verified against the installed `arduino:zephyr` core and real physical hardware testing on the Arduino UNO Q.

Item	Value	Source
Default upload artifact	`{project}.elf-zsk.bin`	`platform.txt` : `upload.extension=elf-zsk.bin`
Static-mode upload artifact	`{project}.bin-zsk.bin`	`boards.txt` : `unoq.menu.link_mode.static.upload.extension`
LED sketch payload size (Dynamic)	75,488 bytes (~74 KB) `.elf-zsk.bin`	`led/build/` listing
LED sketch payload size (Static)	24,788 bytes (~24 KB) `.bin-zsk.bin`	`led/build-static/` listing
Static debug ELF type	`EXEC (Executable file)` , Entry point: `0x08100011`	`readelf -h led.ino_debug.elf`
Static flash addresses	`setup()` at `0x08102590` , `loop()` at `0x0810259c`	`readelf -sW led.ino_debug.elf`
On-board loader match	263,680 bytes read back from flash; 1 byte diff at `0x0803f8bf` (data only, 0 code address shifts)	`dump image` vs `zephyr-*.bin`
Hardware Breakpoints	**Verified working!** Breakpoints hit, call stack traced, and single-stepping confirmed via GDB	Physical board test (GDB + OpenOCD)
Static link mode flags	`build.link_mode=static` , links with `syms-static.ld` , packaged with `zephyr-sketch-tool -prelinked`	`boards.txt` / `platform.txt`
Dynamic link mode flags	Relocatable link (`build-dynamic.ld -r`) against `syms-dynamic.ld` , ELF type `ET_REL`	`platform.txt` / ELF header of `.elf-zsk.bin`
Loader LLEXT settings	`CONFIG_LLEXT=y` , `CONFIG_LLEXT_TYPE_ELF_OBJECT=y` , `CONFIG_LLEXT_HEAP_SIZE=256` , `CONFIG_LLEXT_RODATA_NO_RELOC=y`	`firmwares/zephyr-arduino_uno_q_stm32u585xx.config`
Max sketch flash size	786,432 bytes (768 KB)	`boards.txt` : `unoq.upload.maximum_size`
Max sketch RAM	262,144 bytes (256 KB)	`boards.txt` : `unoq.upload.maximum_data_size`
Sketch flash address	`0x08100000`	`boards.txt` : `unoq.upload.address`
Official upload tool	`remoteocd` (passes loader + sketch over ADB)	`boards.txt` / `platform.txt`

Item	Value	Source
Loader file passed to `remoteocd`	`zephyr-arduino_uno_q_stm32u585xx.elf`, 2,303,728 bytes	`firmwares/` listing
Loader flashed content	`zephyr-arduino_uno_q_stm32u585xx.bin`, 263,680 bytes	`firmwares/` listing
Loader/sketch rewrite policy	Each rewritten only if `flash verify_image` fails	`flash_sketch.cfg`
Post-flash sequence	`reset`, `sleep 100`, `mww 0x40036400 0xCAFFEEEE`, `shutdown`	`flash_sketch.cfg`